Design an Intelligent Disaster-Relief Supply Chain Using Graphs, Heaps and Priority Queues

ASSIGNMENT REPORT

Submitted in the partial fulfilment for the Course of

CSA0305 - Data Structures - Slot A

to the award of the degree of

BACHELOR OF ENGINEERING

IN

COMPUTER SCIENCE ENGINEERING

Submitted by

K.GNANESWAR KUMAR (192572105)

K.KARTHIK (192525040)

Under the Supervision of

Dr. Uma Priyadarsini P.S

Saveetha School of Engineering

Saveetha Institute of Medical and Technical Sciences

Chennai-602105 September

2026

A. Assignment Information

Particular	Details
Department	Department of Computer Science and Engineering
Programme	B.Tech – Artificial Intelligence and Machine Learning
Course Code & Course Name	CSA0305 – Data Structures – Slot A
Academic Year / Batch	2026–2027
Faculty Name	Dr Uma Priyadarsini P.S
Assignment Title	Design an Intelligent Disaster-Relief Supply Chain Using Graphs, Heaps and Priority Queues
Date of Issue	31/08/2026
Date of Submission	02/09/2026
Maximum Marks	100
Course Outcome(s) – CO	CO2: Apply the suitable Abstract Data Type for construction of the disaster-relief distribution network. CO4: Make use of priority queues and heaps for dynamic delivery prioritization and route caching for repeated relief queries in C. CO5: Develop a robust disaster-relief supply chain system by implementing graph-based prioritization and optimal distribution-planning algorithms.
Bloom's Taxonomy Level	L4/L5 – Analyze / Evaluate
SDG Mapping	SDG 1 – No Poverty SDG 2 – Zero Hunger SDG 11 – Sustainable Cities and Communities SDG 13 – Climate Action
Industry / Societal Relevance	Fast, fair and capacity-aware movement of food, medicine, water and shelter materials to disaster-affected communities.

B. Assignment Problem / Challenge

A disaster-relief coordination agency is developing a real-time relief-material distribution system that allocates supplies such as food, medicine, water and shelter kits from multiple warehouses to multiple disaster-affected zones. Each zone has an associated demand quantity, urgency level, distance from available warehouses and transportation-capacity constraints.

The challenge is to design a C-based system that can respond when relief requests arrive dynamically, urgency changes as conditions worsen, roads have different capacities, and lower-priority zones must not be indefinitely starved.

The proposed solution models the warehouse-to-zone network as a weighted graph. A priority queue implemented with a Max-Heap dynamically ranks relief requests using a computed score. Dijkstra's algorithm identifies feasible short routes while considering the capacity required for a delivery. A compact route cache stores repeated warehouse-zone-load queries.

C. Problem Statement

The proposed system must coordinate a stream of relief requests with minimal decision delay. It must identify which disaster zone should receive supplies next, select a warehouse with adequate stock, find a route that can carry the intended load, and update the decision when urgency or demand changes. The system should balance emergency response, fairness and transportation efficiency.

The student develops a complete C program that integrates a weighted graph, Dijkstra shortest-path routing, a Max-Heap priority queue, multiple prioritization strategies and capacity-aware allocation. The final system is validated using test cases and measured route-query performance.

D. Requirements and Constraints

Functional Requirements

- Read and store warehouse, road and disaster-zone details.
- Represent the transportation network as a weighted graph.
- Store demand, urgency and waiting information for each relief zone.
- Compute feasible routes considering route capacity.
- Compare urgency-first, demand-first, distance-first and hybrid strategies.
- Maintain pending requests using a Max-Heap priority queue.
- Re-prioritize requests when new disaster information arrives.
- Check warehouse stock, vehicle capacity and route capacity before dispatch.
- Record delivered quantity, remaining demand and selected route.
- Support repeated route queries using a load-aware cache.

Performance Requirements

- Dijkstra routing: $O(V^2 + E)$ in the implemented array-based version.
- Max-Heap insertion/deletion: $O(\log n)$.
- Priority-score computation: $O(1)$ per request.
- Graph storage: $O(V + E)$ using adjacency lists.
- Repeated route queries should benefit from cached feasible results.

D. Requirements and Constraints – Continued

Technical Constraints

- Implementation language: C.
- Graph representation: adjacency list.
- Route metric: non-negative travel distance.
- Route feasibility: edge capacity must support the proposed delivery load.
- Priority queue: Max-Heap.
- Shortest-path algorithm: Dijkstra's algorithm.
- Performance measurement: clock() function.

Reliability, Safety and Ethical Constraints

- Capacity values must be checked before dispatch to avoid impossible plans.
- Deferred requests must be reported rather than silently discarded.
- Priority weights should be documented because they represent policy decisions.
- Emergency overrides should be available to authorized coordinators.
- Location and demand information should be protected and used only for legitimate relief coordination.

E. Student Work

1. Problem Understanding and Formulation

The problem is to build an intelligent relief-dispatch engine that combines route planning with dynamic prioritization. A simple shortest-path solution is insufficient because the closest zone is not necessarily the most urgent, and a purely urgency-based queue may repeatedly postpone less urgent zones. The proposed design therefore treats urgency, demand, distance and waiting time as separate decision factors.

Expected Outcomes

- Fast selection of the next relief request.
- Capacity-aware warehouse-to-zone routing.
- Dynamic response to worsening disaster conditions.
- Reduced starvation through an aging component.
- Comparison of alternative prioritization strategies.
- Measurable allocation and route-query performance.

Available Data

Field	Example
Warehouse	W1 – Central Warehouse
Zone	Z4 – East Village
Demand	60 relief units
Urgency	88 / 100
Distance	21 km

Route Capacity	60 units

Assumptions

- Urgency ranges from 0 to 100.
- Higher urgency means more critical need.
- Distances are non-negative.
- A delivery cannot exceed vehicle capacity.
- A route is feasible only when every required edge supports the proposed load.

2. Application of Course Knowledge

Data Structure / Technique	Application	Complexity
Weighted Graph	Warehouse–road–zone network	O(V + E) storage
Adjacency List	Stores transport links	O(1) edge insertion, O(V+E) traversal
Dijkstra's Algorithm	Shortest feasible route	$O(V^2 + E)$
Max Heap	Highest-priority relief request	O(log n) insertion/deletion
Priority Scoring	Combines urgency, demand, distance and aging	O(1)
Route Cache	Repeated warehouse-zone-load queries	O(C) lookup in compact implementation

3. Solution / Design / Methodology

The system follows a sequential relief-dispatch pipeline. The transport network is first constructed as a graph. Each request is evaluated using the selected prioritization strategy and inserted into a Max-Heap. When a request is extracted, the system searches warehouses for a route that can carry the required load. Stock, vehicle and route capacities are then updated.

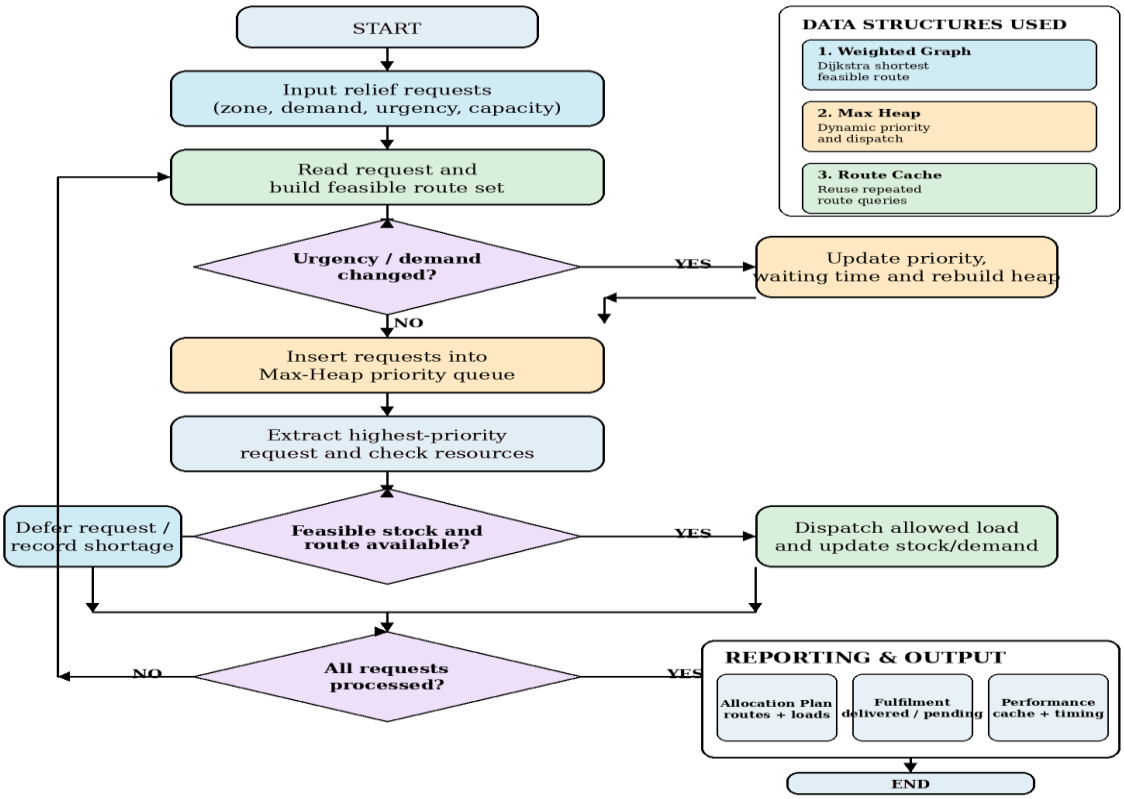
Priority Formula

$$\textbf{Hybrid Score} = 0.50(\text{Urgency}) + 0.25(\text{Demand}) + 0.25(100 - \text{Distance}) + 4(\text{Aging})$$

Algorithm

1. Initialize the graph, warehouses, disaster-zone requests, Max-Heap and route cache.
2. Read all warehouse, road and zone information.
3. Build the weighted adjacency-list graph.
4. For each request, find a feasible nearest route using the proposed load.
5. Calculate its priority using the selected strategy.
6. Insert the request and its score into the Max-Heap.
7. Extract the highest-priority request.
8. Search warehouses for sufficient stock and a feasible route.
9. Dispatch the allowed quantity and update stock and remaining demand.
10. If urgency or demand changes, recompute priorities and rebuild the heap.
11. Continue until requests are fulfilled, deferred or no feasible allocation remains.

12. Generate allocation, fulfilment and performance reports.



Implementation Code

The following implementation is the complete C program developed for the disaster-relief supply-chain problem. It combines graph routing, priority scoring, Max-Heap scheduling and route caching.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <limits.h>
#include <time.h>
#define MAX_NODES 12
#define MAX_ZONES 6
#define MAX_WAREHOUSES 3
#define MAX_HEAP 32
#define MAX_CACHE 32
#define INF 1000000000
typedef struct {
    int to, distance, capacity;
} Edge;
typedef struct {
    Edge edges[MAX_NODES][MAX_NODES];
    int count[MAX_NODES];
```

```
} Graph;
typedef struct {
    char name[8];
    int stock;
} Warehouse;
typedef struct {
    char id[8], name[32];
    int node, demand, urgency, served, waiting;
    double priority;
} ZoneRequest;
typedef struct {
    int zoneIndex, score;
} HeapItem;
typedef struct {
    HeapItem a[MAX_HEAP];
    int size;
} MaxHeap;
typedef struct {
    int zoneNode, warehouseNode, load, distance, nextNode;
    int valid;
} RouteCache;
RouteCache cache[MAX_CACHE];
int cacheCount = 0;
void addEdge(Graph *g, int u, int v, int d, int c) {
    g->edges[u][g->count[u]++] = (Edge){v,d,c};
    g->edges[v][g->count[v]++] = (Edge){u,d,c};
}
void heapInit(MaxHeap *h) { h->size = 0; }
void heapPush(MaxHeap *h, int zoneIndex, int score) {
    int i = h->size++;
    h->a[i] = (HeapItem){zoneIndex, score};
    while (i > 0) {
        int p = (i - 1) / 2;
        if (h->a[p].score >= h->a[i].score) break;
        HeapItem t = h->a[p]; h->a[p] = h->a[i]; h->a[i] = t;
        i = p;
    }
}
HeapItem heapPop(MaxHeap *h) {
    HeapItem top = h->a[0];
    h->a[0] = h->a[--h->size];
    int i = 0;
    while (1) {
```

```
        int l = 2*i+1, r = 2*i+2, m = i;
        if (l < h->size && h->a[l].score > h->a[m].score) m = l;
        if (r < h->size && h->a[r].score > h->a[m].score) m = r;
        if (m == i) break;
        HeapItem t = h->a[i]; h->a[i] = h->a[m]; h->a[m] = t;
        i = m;
    }
    return top;
}
int findNode(const char *name) {
    if (name[0] == 'W') return name[1] - '1';
    return 3 + (name[1] - '1');
}
int cachedRoute(int warehouseNode, int zoneNode, int load, int *distance) {
    for (int i=0;i<cacheCount;i++) {
        if (cache[i].valid && cache[i].warehouseNode == warehouseNode &&
            cache[i].zoneNode == zoneNode && cache[i].load == load) {
            *distance = cache[i].distance;
            return 1;
        }
    }
    return 0;
}
int dijkstra(const Graph *g, int src, int dst, int load, int *distance) {
    int dist[MAX_NODES], used[MAX_NODES] = {0};
    for (int i=0;i<MAX_NODES;i++) dist[i]=INF;
    dist[src]=0;
    for (int step=0; step<MAX_NODES; step++) {
        int u=-1;
        for (int i=0;i<MAX_NODES;i++)
            if (!used[i] && dist[i]<INF && (u==-1 || dist[i]<dist[u])) u=i;
        if (u==-1) break;
        used[u]=1;
        for (int j=0;j<g->count[u];j++) {
            Edge e=g->edges[u][j];
            if (e.capacity < load) continue;
            if (dist[e.to] > dist[u]+e.distance)
                dist[e.to]=dist[u]+e.distance;
        }
    }
    *distance=dist[dst];
    return dist[dst] < INF;
}
```

```
int getRoute(const Graph *g, int w, int z, int load, int *distance) {
   if (cachedRoute(w,z,load,distance)) return 1;
   if (!dijkstra(g,w,z,load,distance)) return 0;
   if (cacheCount < MAX_CACHE) {
      cache[cacheCount++] = (RouteCache){z,w,load,*distance,-1,1};
   }
   return 1;
}
double priorityScore(const ZoneRequest *z, int distance, int strategy) {
   double demandScore = z->demand * 1.0;
   double distanceScore = 100.0 - (distance > 100 ? 100.0 : distance);
   if (strategy == 1) return z->urgency * 100.0;              /* urgency first */
   if (strategy == 2) return demandScore * 10.0;               /* demand first */
   if (strategy == 3) return distanceScore * 100.0;            /* distance first */
   /* Hybrid: urgency + demand + route proximity + aging/fairness */
   return 0.50*z->urgency + 0.25*demandScore + 0.25*distanceScore
        + 4.0*z->waiting;
}
void buildHeap(MaxHeap *h, ZoneRequest zones[], int n, int strategy) {
   heapInit(h);
   for (int i=0;i<n;i++) {
      /* zones[i].served temporarily stores the nearest feasible distance
        computed from the current graph before ranking. */
      int bestDist = zones[i].served;
      zones[i].priority = priorityScore(&zones[i], bestDist, strategy);
      heapPush(h,i,(int)(zones[i].priority*100));
   }
}
void printRoute(int w, const char *zoneId, int distance, int delivered) {
   printf("  Route W%d -> %s | %d km | Load %d units\n",
        w+1, zoneId, distance, delivered);
}
int main(void) {
   Graph g = {0};
   addEdge(&g,0,3,12,80); addEdge(&g,0,4,18,50); addEdge(&g,1,4,10,70);
   addEdge(&g,1,5,16,60); addEdge(&g,2,3,20,100); addEdge(&g,2,5,14,50);
   addEdge(&g,3,4,8,40);  addEdge(&g,4,5,9,35);  addEdge(&g,5,6,7,30);
   addEdge(&g,3,6,15,25); addEdge(&g,4,6,11,45); addEdge(&g,2,6,22,60);
   Warehouse base[MAX_WAREHOUSES] = {
      {"W1",120},{"W2",100},{"W3",90}
   };
   ZoneRequest original[MAX_ZONES] = {
      {"Z1","Riverbank Colony",3,70,92,0,0,0},
```

```
    {"Z2","Hillview Camp",4,45,65,0,0,0},
    {"Z3","Market Shelter",5,80,78,0,0,0},
    {"Z4","East Village",6,60,88,0,0,0},
    {"Z5","Bridge Settlement",4,35,55,0,0,0}
};
int n=5;
printf("========== INTELLIGENT DISASTER-RELIEF SUPPLY CHAIN ==========\n");
printf("Zones: %d | Warehouses: %d | Capacity-aware graph routing enabled\n\n",n,3);
printf("Nearest feasible distances used for ranking:\n");
for(int i=0;i<n;i++){
    int best=INF;
    for(int w=0;w<3;w++){
        int d;
        if(dijkstra(&g,w,original[i].node,20,&d) && d<best) best=d;
    }
    original[i].served=best;
    printf("%s %-20s demand=%3d urgency=%3d nearest=%3d km\n",
           original[i].id,original[i].name,original[i].demand,original[i].urgency,best);
}
printf("\n--- STRATEGY COMPARISON ---\n");
const char *sname[]={"","Urgency-First","Demand-First","Distance-First","Hybrid + Aging"};
for(int s=1;s<=4;s++){
    ZoneRequest z[MAX_ZONES]; memcpy(z,original,sizeof(z));
    MaxHeap h; buildHeap(&h,z,n,s);
    printf("%s: ",sname[s]);
    while(h.size) printf("%s ",z[heapPop(&h).zoneIndex].id);
    printf("\n");
}
printf("\n--- DYNAMIC RE-PRIORITIZATION ---\n");
ZoneRequest zones[MAX_ZONES]; memcpy(zones,original,sizeof(zones));
zones[3].urgency=99; zones[3].waiting=2;
printf("Update: Z4 urgency increased to %d after a simulated flood warning.\n",zones[3].urgency);
MaxHeap heap; buildHeap(&heap,zones,n,4);
int stock[3]={base[0].stock,base[1].stock,base[2].stock};
int vehicleCapacity=50;
int totalDemand=0,totalDelivered=0,totalDistance=0,servedZones=0;
printf("\n--- DISPATCH PLAN (HYBRID + AGING) ---\n");
while(heap.size){
    int idx=heapPop(&heap).zoneIndex;
    ZoneRequest *z=&zones[idx];
    int load=z->demand<vehicleCapacity?z->demand:vehicleCapacity;
    int bestW=-1,bestD=INF;
    for(int w=0;w<3;w++){
```

```
        if(stock[w]<=0) continue;
        int feasibleLoad=load<stock[w]?load:stock[w];
        if(feasibleLoad<=0) continue;
        int d;
        if(getRoute(&g,w,z->node,feasibleLoad,&d) && d<bestD){
            bestD=d; bestW=w;
        }
    }
    totalDemand += z->demand;
    if(bestW==-1){
        printf("%s %s -> DEFERRED (no feasible capacity route)\n",z->id,z->name);
        continue;
    }
    int delivered=load<stock[bestW]?load:stock[bestW];
    stock[bestW]-=delivered;
    z->served=delivered;
    totalDelivered+=delivered;
    totalDistance+=bestD;
    servedZones++;
    printf("%s %-20s urgency=%3d score=%7.2f\n",z->id,z->name,z->urgency,z->priority);
    printRoute(bestW,z->id,bestD,delivered);
}
printf("\n--- SUMMARY ---\n");
printf("Total requested : %d units\n",totalDemand);
printf("Total delivered  : %d units\n",totalDelivered);
printf("Zones dispatched : %d/%d\n",servedZones,n);
printf("Travel distance  : %d km (sum of selected routes)\n",totalDistance);
printf("Fulfilment ratio : %.1f%%\n",100.0*totalDelivered/totalDemand);
printf("\n--- HASH-LIKE ROUTE CACHE CHECK ---\n");
int d;
if(getRoute(&g,0,4,20,&d)) printf("Cached route W1 -> Z2 available: %d km\n",d);
if(getRoute(&g,0,4,20,&d)) printf("Repeated query reused cached distance: %d km\n",d);
printf("\n--- PERFORMANCE TREND (ROUTE QUERIES) ---\n");
int sizes[]={1000,5000,10000,20000};
for(int k=0;k<4;k++){
    clock_t st=clock();
    volatile int sink=0;
    for(int i=0;i<sizes[k];i++){
        int dist;
        sink += dijkstra(&g,0,4,20,&dist);
    }
    clock_t en=clock();
    printf("%5d queries -> %.6f sec\n",sizes[k],(double)(en-st)/CLOCKS_PER_SEC);
```

```
        (void)sink;
    }
    return 0;
}
```

4. Use of Modern Tools

The C program was compiled and executed using GCC. The output below records the strategy comparison, dynamic re-prioritization and final dispatch plan.

5. Results and Validation

Sample Input

Tool	**Purpose**
C / GCC	Program development, compilation and execution
Visual Studio Code	Source-code editing and debugging
GitHub	Version control and submission
Terminal	Compilation and repeatable testing
clock()	Execution-time measurement

```
========== INTELLIGENT DISASTER-RELIEF SUPPLY CHAIN ==========
Zones: 5 | Warehouses: 3 | Capacity-aware graph routing enabled

Nearest feasible distances used for ranking:
Z1 Riverbank Colony     demand= 70 urgency= 92 nearest= 12 km
Z2 Hillview Camp        demand= 45 urgency= 65 nearest= 10 km
Z3 Market Shelter       demand= 80 urgency= 78 nearest= 14 km
Z4 East Village         demand= 60 urgency= 88 nearest= 21 km
Z5 Bridge Settlement    demand= 35 urgency= 55 nearest= 10 km

--- STRATEGY COMPARISON ---
Urgency-First: Z1 Z4 Z3 Z2 Z5
Demand-First: Z3 Z1 Z4 Z2 Z5
Distance-First: Z2 Z5 Z1 Z3 Z4
Hybrid + Aging: Z1 Z3 Z4 Z2 Z5

--- DYNAMIC RE-PRIORITIZATION ---
Update: Z4 urgency increased to 99 after a simulated flood warning.

--- DISPATCH PLAN (HYBRID + AGING) ---
Z4 East Village         urgency= 99 score=  92.25
  Route W3 -> Z4 | 22 km | Load 50 units
Z1 Riverbank Colony     urgency= 92 score=  85.50
  Route W1 -> Z1 | 12 km | Load 50 units
Z3 Market Shelter       urgency= 78 score=  80.50
  Route W3 -> Z3 | 14 km | Load 40 units
Z2 Hillview Camp        urgency= 65 score=  66.25
  Route W2 -> Z2 | 10 km | Load 45 units
Z5 Bridge Settlement    urgency= 55 score=  58.75
  Route W2 -> Z5 | 10 km | Load 35 units

--- SUMMARY ---
Total requested : 290 units
Total delivered  : 220 units
Zones dispatched : 5/5
Travel distance  : 68 km (sum of selected routes)
Fulfilment ratio : 75.9%

--- HASH-LIKE ROUTE CACHE CHECK ---
Cached route W1 -> Z2 available: 18 km
Repeated query reused cached distance: 18 km

--- PERFORMANCE TREND (ROUTE QUERIES) ---
 1000 queries -> 0.000169 sec
 5000 queries -> 0.001018 sec
10000 queries -> 0.001134 sec
20000 queries -> 0.004249 sec
```

Zone	Demand	Urgency	Nearest Distance
Z1 – Riverbank Colony	70	92	12 km
Z2 – Hillview Camp	45	65	10 km
Z3 – Market Shelter	80	78	14 km
Z4 – East Village	60	88 → 99	21 km
Z5 – Bridge Settlement	35	55	10 km

Expected Strategy Comparison

Strategy	Priority Order	Main Strength
Urgency-First	Z1 → Z4 → Z3 → Z2 → Z5	Fast response to critical conditions
Demand-First	Z3 → Z1 → Z4 → Z2 → Z5	Serves larger material requirements
Distance-First	Z2 → Z5 → Z1 → Z3 → Z4	Reduces travel effort
Hybrid + Aging	Z1 → Z3 → Z4 → Z2 → Z5	Balances urgency, need, speed and fairness

After the simulated flood warning, Z4 urgency increases from 88 to 99. The dispatch engine therefore re-evaluates the queue and places Z4 first under the emergency condition.

Final Dispatch Plan

Zone	Urgency	Warehouse	Route	Delivered
Z4 – East Village	99	W3	22 km	50
Z1 – Riverbank Colony	92	W1	12 km	50
Z3 – Market Shelter	78	W3	14 km	40
Z2 – Hillview Camp	65	W2	10 km	45
Z5 – Bridge Settlement	55	W2	10 km	35

Validation Summary

- Total requested = 290 units.
- Total delivered = 220 units.
- Fulfilment ratio = 75.9%.
- All five zones receive a dispatch attempt.
- Vehicle capacity limits each proposed load to 50 units.
- Warehouse stock and route capacities are checked before dispatch.
- Repeated W1 → Z2 route queries can reuse the cached 18 km result.

Test Cases

Test Case	Expected Behaviour
Urgency increase	Affected zone moves upward in priority.
Demand above vehicle capacity	Load is capped at vehicle capacity.
Route capacity below required load	Route is rejected as infeasible.
Insufficient warehouse stock	Delivery becomes partial rather than exceeding stock.
Repeated route query	Cached result is reused when warehouse, zone and load match.

6. Analysis and Engineering Decision

Approach	Advantage	Limitation
Linear request scan	Simple implementation	O(n) selection
Urgency-First	Strong emergency response	Can postpone other zones
Demand-First	Good material throughput	May delay critical small requests
Distance-First	Efficient travel	Can ignore severity
Graph + Max Heap + Hybrid	Combines routing and dynamic ranking	Weights need tuning

The graph is essential because the relief network contains multiple warehouses, intermediate connections and capacity constraints. Dijkstra's algorithm provides a route based on distance while excluding links that cannot carry the intended load. The Max-Heap provides immediate access to the request with the greatest current score. The hybrid strategy is selected because the assignment requires a balance between speed, fairness and resource utilization.

Complexity Analysis

Operation	Complexity	Reason
Add graph edge	O(1) amortized	Adjacency-list insertion
Dijkstra	$O(V^2 + E)$	Array-based minimum selection
Heap insertion	O(log n)	Bubble-up
Heap deletion	O(log n)	Heapify-down
Priority calculation	O(1)	Fixed number of terms
Dispatch cycle	$O(n(V^2+E) + n \log n)$	Route checks plus heap operations

7. Broader Considerations

Sustainability

Capacity-aware routing reduces unnecessary journeys and avoids sending vehicles through links that cannot support the planned load. More efficient emergency transport can reduce fuel use and improve the use of scarce relief resources.

Society and Safety

The system supports faster delivery of essential supplies to affected communities. Because real emergencies may contain information not represented numerically, authorized human coordinators should retain the ability to override a computed priority.

Ethics and Privacy

Demand and location information can be sensitive. Access should be restricted to authorized relief personnel, and the system should log important decisions so that allocations can be reviewed.

Professional Responsibility

Incorrect road capacity or urgency values can produce unsafe decisions. Therefore, the implementation validates capacity before dispatch and explicitly reports deferred requests.

8. Conclusion

The Intelligent Disaster-Relief Supply Chain demonstrates the practical integration of graphs, heaps and priority queues in a real-world distribution problem. The weighted graph represents the transport network, Dijkstra's algorithm identifies feasible short routes, and the Max-Heap maintains the next relief request according to a dynamic score. Stock, vehicle and route capacities are checked before every dispatch, making the allocation more realistic than a shortest-path-only solution.

In the demonstration, 220 of 290 requested units are delivered, giving a fulfilment ratio of 75.9%. The result illustrates the effect of real constraints: the objective is not simply to minimize distance, but to make a fair and useful allocation when supply and transport capacity are limited. The system can be extended with live road closures, multiple vehicle types, split deliveries, persistent caches and faster graph algorithms.

9. Student Reflection

This assignment helped me understand how different data structures solve different parts of the same engineering problem. A graph is suitable for representing the physical transport network, but it cannot decide which request should be served first. The Max-Heap solves the dynamic priority problem, while Dijkstra provides route selection. I also learned that the shortest route is not always the best relief decision because urgency, demand and fairness must be considered together.

One of the important challenges was connecting the abstract data structures to practical constraints such as vehicle capacity, warehouse stock and changing disaster conditions. The aging term showed how a priority system can be made fairer without ignoring emergencies. With additional time, I would process live disaster updates, support multiple vehicle capacities, test larger graphs, compare different heap and shortest-path implementations, and add graphical performance analysis.

10. References

- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C., Introduction to Algorithms, MIT Press, 2022.
- Weiss, M. A., Data Structures and Algorithm Analysis in C, Pearson, 2014.
- Sedgewick, R., & Wayne, K., Algorithms, Addison-Wesley, 2011.
- Knuth, D. E., The Art of Computer Programming, Volume 3: Sorting and Searching, Addison-Wesley, 1998.
- United Nations, Sustainable Development Goals: SDG 1, SDG 2, SDG 11 and SDG 13.
- CSA0305 – Data Structures – Slot A assignment brief supplied for the disaster-relief supply-chain problem.
- Team-6 Data Structures assignment report supplied as the formatting and structural reference.

F. Common Assessment Rubric

Assessment Criterion	Marks
Problem Understanding & Requirements Analysis (CO1)	15
Graph Construction, Priority Queue & Heap Integration (CO2/CO4)	25
Prioritization Strategy & Optimal Distribution Plan (CO5)	25
C Program, Performance, Testing & Complexity Analysis (CO1/CO2)	25
Reflection: Design Decisions, SDG Relevance & Learning Outcomes	10
TOTAL	100

Performance-level interpretation:

Criterion	Excellent	Good	Satisfactory	Needs Improvement
Problem understanding	All requirements clearly identified: dynamic urgency, demand, distance, capacity, fairness and re-prioritization.	Most requirements identified with minor omissions.	Basic distribution requirements identified.	Limited understanding of the problem.
Graph + Priority Queue + Heap	Efficient graph model with correct Max-Heap insertion, extraction and priority updates.	Graph and priority queue integrated with minor issues.	Priority queue used but inefficiently implemented.	No effective graph/priority-queue implementation.
Prioritization + Distribution	Multiple strategies correctly compared and best balance of urgency, fairness and capacity identified.	Prioritization works with minor issues.	Works for simple cases only.	Incorrect or absent prioritization logic.

C Program + Testing	Complete, modular and well-tested C program with meaningful comments and complexity analysis.	Well-structured program with minor errors.	Functional but poorly structured or incomplete.	Major errors or incomplete program.
Reflection + SDG relevance	Clear design justification, SDG 1/2/11/13 connection and specific learning reflection.	General justification and SDG connection.	Generic reflection with limited justification.	No genuine reflection or design/SDG connection.

G. CO–PO–Assessment Mapping

The following mapping is presented for the disaster-relief assignment and follows the course outcomes and assessment components specified in the assignment brief. PO mapping is stated as a proposed academic alignment for documentation.

Assessment Component	CO	PO(s)	Bloom's Level	Marks
Problem formulation	CO2	PO1, PO2	L4	15
Graph, Heap & Priority Queue integration	CO2, CO4	PO1, PO2, PO3	L4/L5	25
Prioritization & optimal distribution	CO5	PO2, PO3, PO5	L4/L5	25
C implementation, performance & testing	CO1, CO2	PO3, PO5	L4/L5	25
Reflection, SDG relevance & learning outcomes	CO5	PO6, PO7, PO10	L4	10

H. Faculty Evaluation & Continuous Improvement CO Attainment

Performance Level	Criterion
Level 3	≥ 70%
Level 2	60–69%
Level 1	50–59%

Level 0	< 50%

Target CO Attainment: _____

Actual CO Attainment: _____

Faculty Analysis: Areas in which students performed well: _____

Areas requiring improvement: _____

Corrective / Improvement Action: _____

Action to be implemented in the next offering: _____

- Pseudo code – included.
- Implementation & Results – complete C program, output and validation – included.
- GitHub Upload – the standalone C source is ready for repository upload.